

Young Innovators

Obstacle Avoiding Robot with Scanning Sonar

Look left, look right, then choose the clearest path

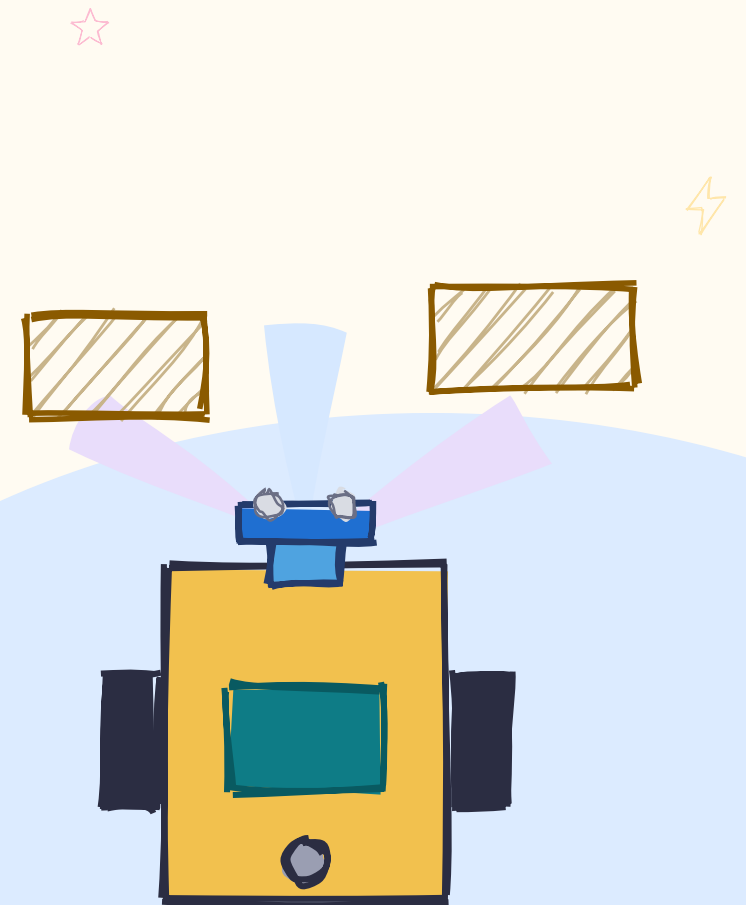
Ultrasonic radar scan

L298N motor control

Functions

Did you know?

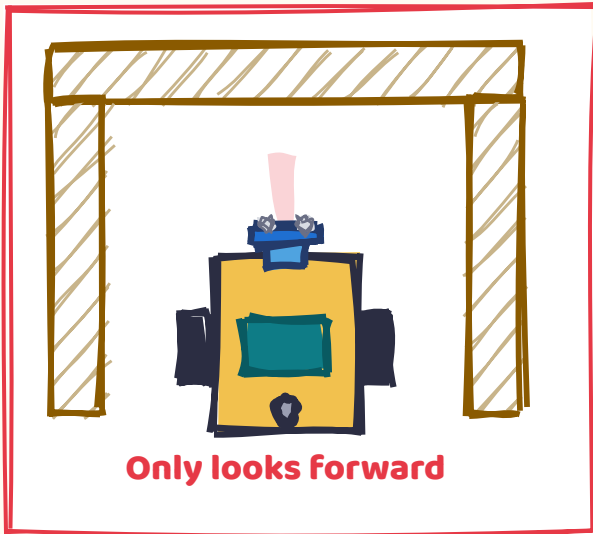
Robot vacuum cleaners map a whole house by bumping and scanning, then plan the fastest cleaning route.



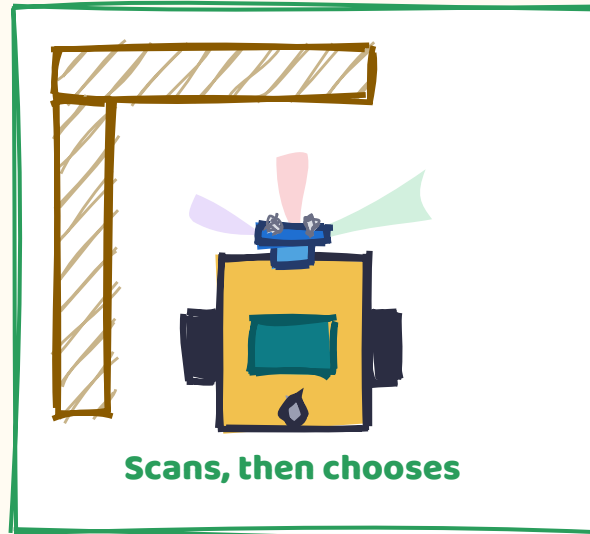


Let's think...

Why does a simple robot get stuck?



Only looks forward



Scans, then chooses

The problem

A robot that only sees straight ahead turns **randomly** when blocked. It can turn straight into another wall, or get trapped in a corner forever.



Today's mission

Put the sensor on a **servo neck**. When blocked: stop, **look left and right**, measure both, and turn towards the **clearer** side.

Think

How do YOU decide which way to go in a crowded corridor?





Learning aims

By the end of this session, I can...



1

Use a servo to **scan** an ultrasonic sensor in three directions

2

Control two DC motors' **direction and speed** with an L298N

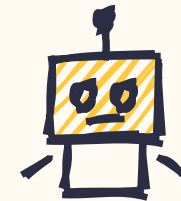
3

Write and reuse **functions**: forward, stop, scan, turn

4

Make decisions by **comparing** distances

5

**Tune and test** a robot until it beats an obstacle course**Keywords**autonomous • sonar • servo • H-bridge
• PWM • function • differential drive



Real world

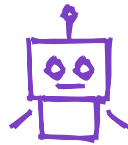


Machines that sense before they move



Self-driving cars

Use lidar, radar, cameras and ultrasonic sensors together



Robot vacuums

Bump sensors plus a spinning laser scanner



Warehouse robots

Carry shelves around without hitting each other



Parking sensors

The beeping in cars uses the same ultrasonic idea



A **lidar** on a self-driving car is a scanning sensor too. It just spins very fast and uses **laser light** instead of sound.

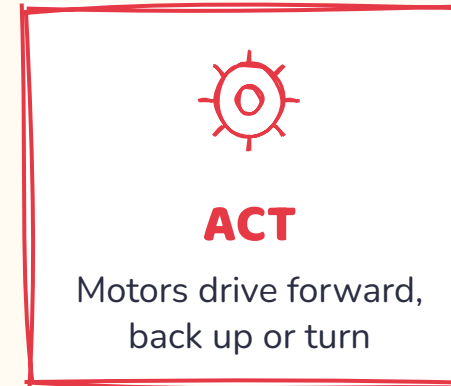
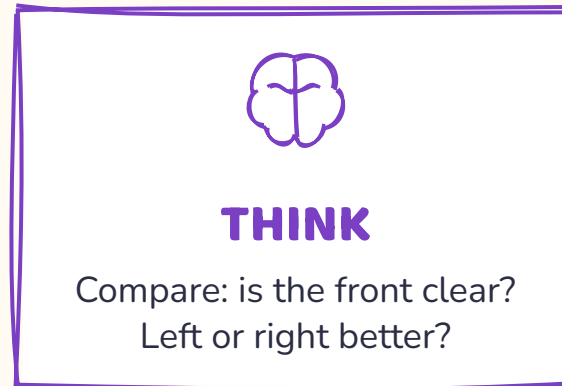
Same idea, bigger scale
Our servo + sonar is a tiny, slow lidar!





How autonomous robots work

Sense, think, act... repeat!



Autonomous means the robot makes its own decisions. Nobody is holding a remote control.

Next class

We add a remote control, and compare which is harder to build!





Stand up and play!

Human robot with a sonar partner



How to play (pairs, cleared space)

1. The **robot** closes their eyes and walks **slowly**, hands out in front.
2. The **sonar** walks behind and may only say a **distance**: "2 metres", "50 cm".
3. Under 1 metre: robot **stops**. Sonar looks left and right, says both numbers.
4. The robot (not the sonar!) **decides** which way to turn.

Talk about it

Was it hard to choose with only numbers?
What extra information did you wish you had? That is exactly the robot's life!



Notice

You stopped, scanned, compared, then turned. That is our algorithm!



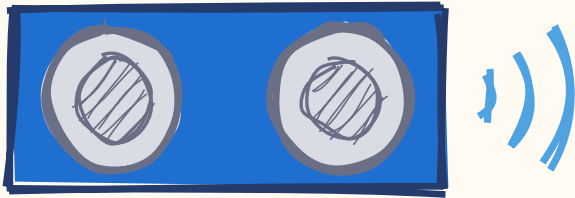
Safety: walk slowly, teacher clears the space, the sonar keeps a hand on the robot's shoulder.





Know your sensor

When sonar gets fooled

**HC-SR04**

Quick reminder

Range **2 to 400 cm**. distance (cm) = echo time (μ s) $\div$ 58. The beam is a cone about **15°** wide.



Flat wall, straight on
Echo comes straight back



Wall at a sharp angle
Sound bounces away: no echo!



Thin chair leg
Too thin for the 15° cone



Curtain or cushion
Soft things swallow sound



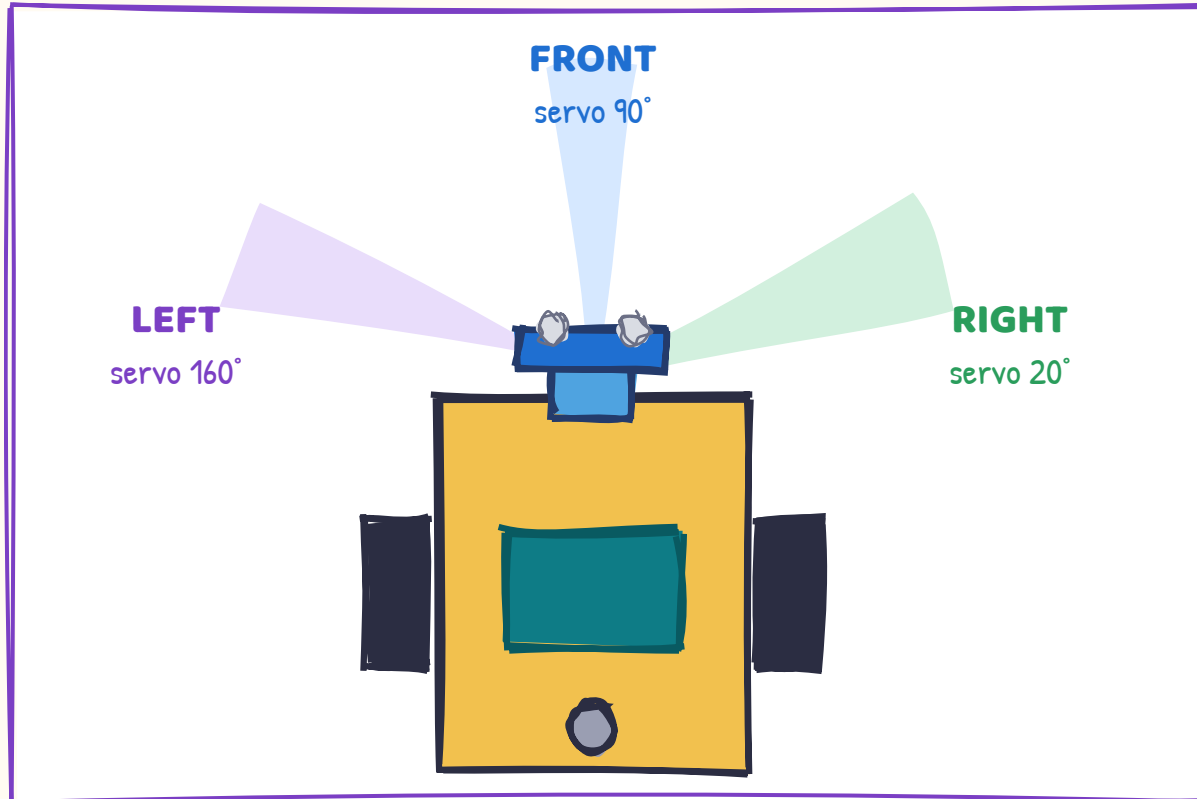
Engineer's trick: if no echo comes back, **pulseIn** returns 0. We treat 0 as "far away", not as "touching"!





The scanning neck

Turn the sensor, not the robot



How the scan works

1. `neck.write(160)`: turn the sensor left.
2. **Wait 400 ms** so the servo arrives.
3. Measure the distance.
4. Repeat for the right, then face front again.



Why wait?

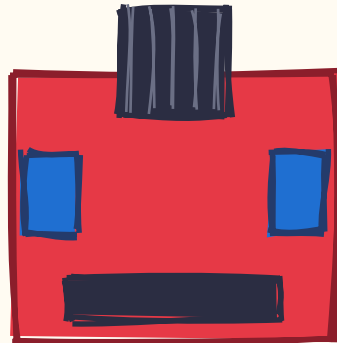
A servo needs time to move. Measure too early and you measure the wrong direction!

Your servo may be mounted the other way round. Test which angle really points left!

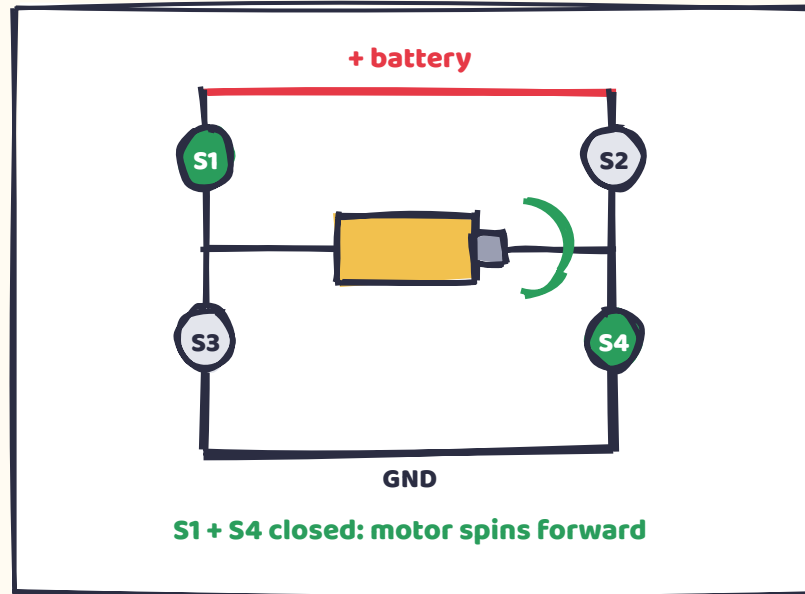


Drive the wheels

The L298N motor driver (an H-bridge)



L298N driver



IN1	IN2	Left motor
HIGH	LOW	Forward
LOW	HIGH	Backward
LOW	LOW	Stop

Speed with PWM

ENA (left) and ENB (right) take `analogWrite(0 to 255)`. 255 = full speed. Below about 100, many motors just hum.

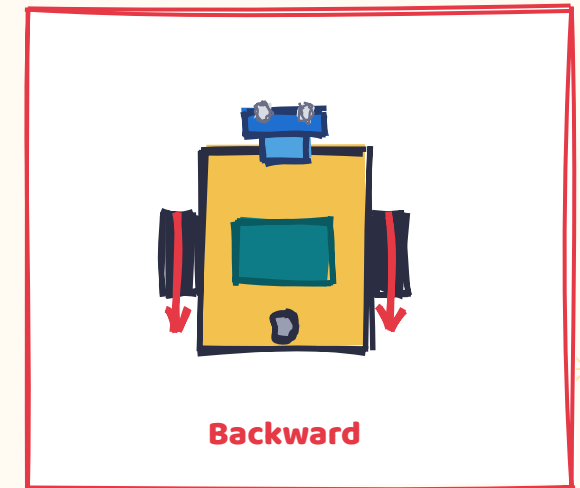
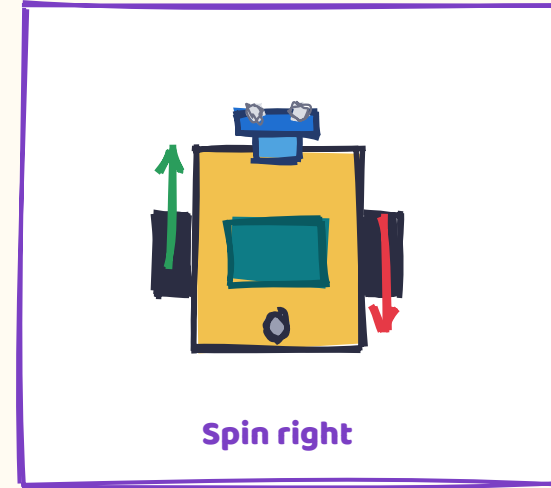
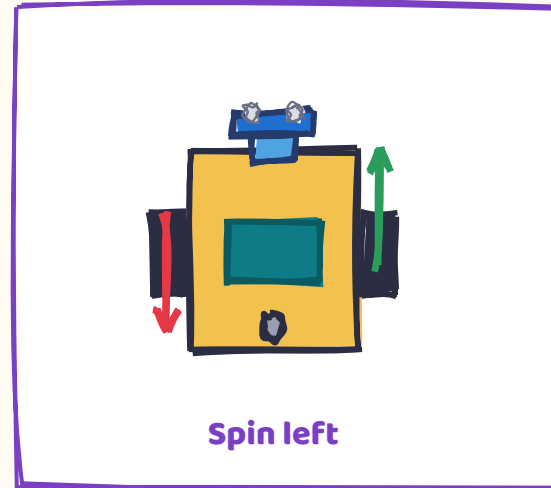
Why an H-bridge?

A motor reverses when you **swap** its + and -. Four switches shaped like an **H** let the Arduino do that swap electronically.



Steering without a steering wheel

Differential drive



The rule

Both wheels the same way = straight. Wheels **opposite** ways = **spin on the spot**. The longer you spin, the bigger the turn.

Tank trick

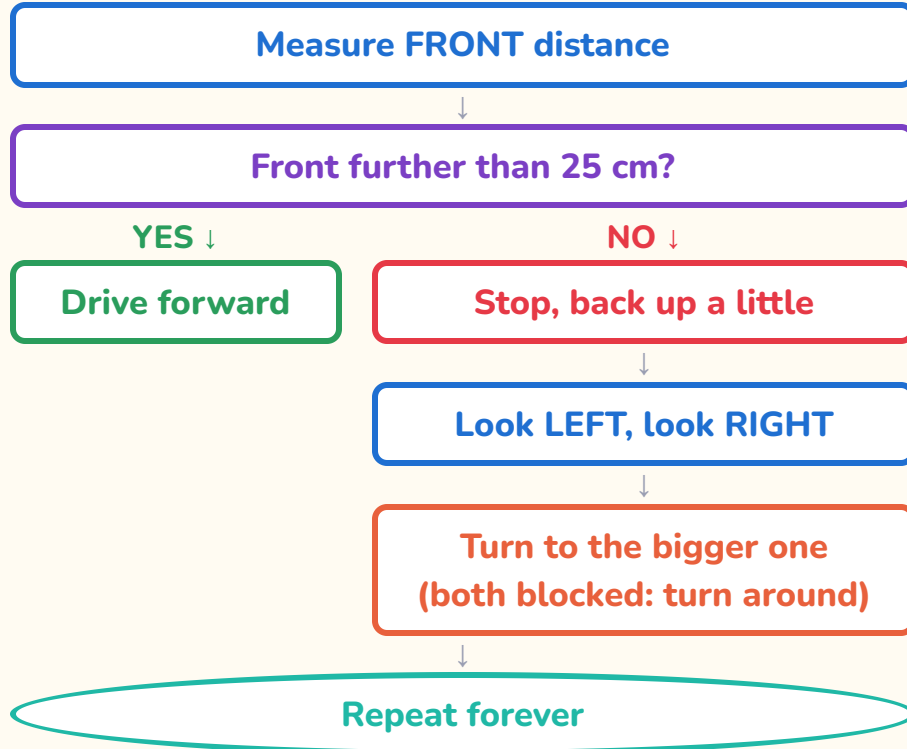
Tanks and bulldozers steer exactly like this!





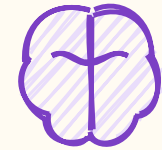
Think

The decision algorithm



Why back up first?

If the robot spins while touching the box, a wheel can catch on it. A short reverse gives it room to turn.



Why 25 cm?

The robot needs **stopping distance**. Faster robot = bigger safety distance. You will tune this!

Pseudocode first
Engineers write the plan in words before writing code.





Build it

Our robot kit (per group)



Arduino Uno

The brain



2WD chassis kit

2 motors, wheels, caster



L298N driver

Powers the motors



HC-SR04

The sonar eye



SG90 servo

The scanning neck



Battery pack

Motor power



Assembly tips

Mount the servo at the **front centre**, sensor eyes facing forward at 90°. Keep wires away from the wheels with cable ties. Batteries **low** and in the middle so the robot does not tip.

Screwdriver team

One builds, one reads the instructions, one checks.
Swap every 10 minutes.

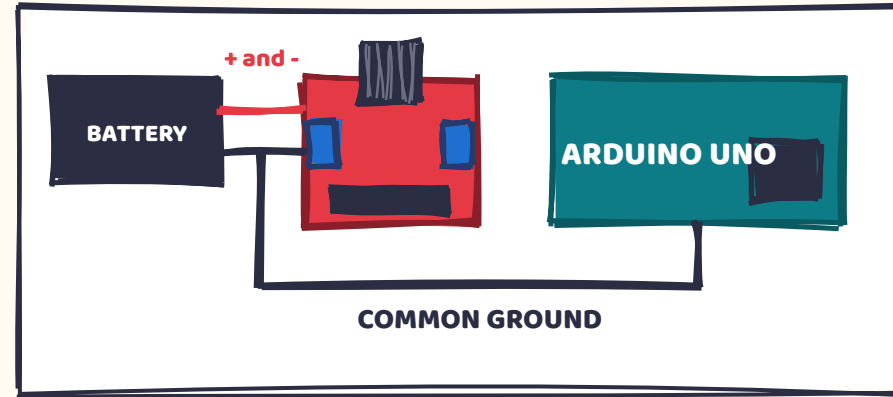


Build it

Wiring and power

Connection Arduino pin

ENA / ENB	5 / 6 (PWM)
IN1 / IN2	7 / 8
IN3 / IN4	4 / 2
Servo signal	9
Sonar Trig / Echo	12 / 11
Sonar + servo VCC	5V
L298N GND	Arduino GND!



Golden rule

Connect the grounds.
Without a shared GND the L298N cannot understand the Arduino's signals.

Why separate power?

Motors pull big, spiky currents. Running them from the Arduino's 5V pin causes **resets**. Motors get their own battery; the Arduino gets USB or its own pack.

Pin 9 note

The Servo library stops PWM on pins 9 and 10. That is why ENA and ENB use 5 and 6.



Code it, part 1

The main program (Arduino IDE)



```
#include <Servo.h>
const int ENA = 5, IN1 = 7, IN2 = 8, IN3 = 4, IN4 = 2, ENB = 6;
const int TRIG = 12, ECHO = 11;
Servo neck;
const int SAFE = 25;          // cm: tune this!
int SPEED = 180;              // 0 to 255

void setup() {
  int pins[] = {ENA, IN1, IN2, IN3, IN4, ENB, TRIG};
  for (int p : pins) pinMode(p, OUTPUT);
  pinMode(ECHO, INPUT);
  neck.attach(9); neck.write(90); // face forward
  delay(1000);
}

void loop() {
  if (distance() > SAFE) {
    forward();
  } else {
    backward(); delay(300); stopCar(); // back off a little
    int left = look(160); int right = look(20);
    neck.write(90); delay(300);
    if (left > right && left > SAFE) turnLeft(400);
    else if (right > SAFE) turnRight(400);
    else turnRight(800); // turn around
  }
}
```

Read the loop

Look how short **loop()** is! It reads like the flowchart because the hard work lives in **functions**: **distance()**, **look()**, **forward()**, **turnLeft()**...

left > right && left > SAFE: left is better AND actually clear.

Tune here

SAFE, SPEED and the turn times (400, 800 ms) are YOUR numbers to test and change.





Code it, part 2

The functions: build once, use many times



```
int distance() {
    digitalWrite(TRIG, LOW); delayMicroseconds(2);
    digitalWrite(TRIG, HIGH); delayMicroseconds(10);
    digitalWrite(TRIG, LOW);
    long t = pulseIn(ECHO, HIGH, 30000); // give up after 30 ms
    if (t == 0) return 400;               // no echo = far away
    return t / 58;
}

int look(int angle) {
    neck.write(angle); delay(400);        // let the servo arrive
    return distance();
}

void drive(int a, int b, int c, int d) {
    analogWrite(ENA, SPEED); analogWrite(ENB, SPEED);
    digitalWrite(IN1, a); digitalWrite(IN2, b);
    digitalWrite(IN3, c); digitalWrite(IN4, d);
}

void forward() { drive(HIGH, LOW, HIGH, LOW); }
void backward() { drive(LOW, HIGH, LOW, HIGH); }
void stopCar() { drive(LOW, LOW, LOW, LOW); }
void turnLeft(int ms) { drive(LOW, HIGH, HIGH, LOW); delay(ms); stopCar(); }
void turnRight(int ms) { drive(HIGH, LOW, LOW, HIGH); delay(ms); stopCar(); }
```

Why functions?

drive() is written once. **forward()**, **backward()** and the turns just call it with different HIGH / LOW patterns.

Fix a bug in one place, and it is fixed everywhere.

Return values

distance() and **look()** RETURN a number. void functions just DO something.





Test like an engineer

Tune your robot



Setting	Start	Your best	What to watch
SAFE distance	25 cm	_____	Too small: crashes. Too big: turns at nothing
SPEED	180	_____	Too slow: motors hum and stall
Turn time	400 ms	_____	Measure: how many ms for a 90° turn?
Scan wait	400 ms	_____	Too short: measures the wrong way

One change at a time

Change **one** number, test 3 times, write the result. Change two at once and you never know which one helped.



Calibrate the 90° turn: put tape on the floor, run turnRight(ms) with different values until the robot turns exactly a quarter circle.





Debug like a pro

Robot not behaving?



Robot spins instead of going forward

One motor is wired backwards. Swap its two wires on the L298N.



Robot goes backwards

Swap both motors' wires, or swap HIGH / LOW in forward().



Arduino resets when motors start

Motors on the Arduino 5V? Use a separate battery and a common GND.



Distance always 400

Trig and Echo swapped, or nothing in range.



Servo twitches and shakes

Weak power. Fresh batteries; power the servo from a stable 5V.

Lift it first!

Test motor code with the robot **upside down** or on a box, wheels in the air. No runaway robots!

Serial is your friend

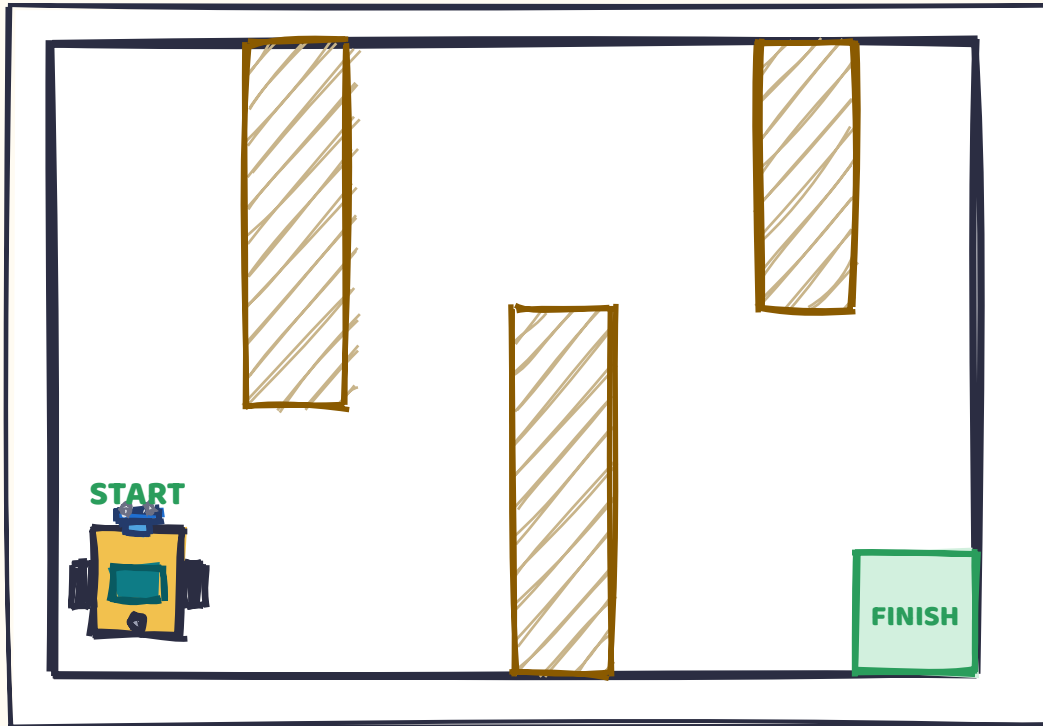
Print left, front and right every scan.
Numbers tell the truth.





The challenge

Beat the obstacle course!



Scoring

Time to finish	seconds
Each wall touch	+ 5 s penalty
Hand help	+ 15 s penalty
Did not finish	distance reached



More challenges

Scan **5 angles** instead of 3 · slow down as walls get closer · beep faster the nearer the obstacle (like a car).

Fastest wins?
Only if it never touches a wall!



Show what you know

What did we learn?

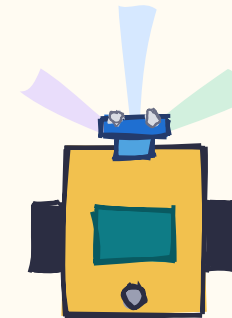
- ✓ Autonomous robots repeat: sense, think, act
- ✓ A servo can scan one sensor in many directions
- ✓ An H-bridge reverses a motor; PWM sets its speed
- ✓ Wheels in opposite directions = spin on the spot
- ✓ Functions keep code short, readable and reusable



Exit ticket: left reads 18 cm, right reads 70 cm, SAFE is 25.
What does the robot do?

At home

Watch a robot vacuum or car parking sensor video. Which sensors can you spot?



Next: a
Bluetooth Robot
Car with Phone
App!